Architecture of the Proposed Rule-Based Knowledge Graph Completion System Using CoDEx-M

1. Input and data preparation

Test split
test.txt 10,311 positives
test_negatives.txt 10,311 supplied negatives

Display labels
entity_labels.json · relation_labels.json
(presentation only; IDs are processing keys)

Training split
data/train.txt
185,584 triples · 17,050 entities · 51 relations

Validation split
valid.txt 10,310 positives
valid_negatives.txt 10,310 supplied negatives

structural checks only

Phase 1: Dataset inspection and split verification
row format, duplicates, ID checks, label coverage,
zero overlap between positive splits (phase1_dataset_report.md)

train.txt + labels (built independently of Phase 2)

train.txt only

2. Graph and ontology construction (training data only)

Phase 3: RDF/OWL ontology dataset (phase3_ontology.py)
codexm_train_ontology.trig — three named graphs:
train (185,584) · labels (51,303) · provenance (23)
owl:NamedIndividual / owl:ObjectProperty only; no domain/range axioms

Phase 2: Training knowledge graph (phase2_graph.py)
NetworkX MultiDiGraph: 17,050 entity nodes,
185,584 relation-labelled directed edges

training named graph

training_graph.graphml · graph_statistics.json
interactive_graph.html (entity search, neighbourhood, relation filter)

GraphML training facts

not used in scoring yet

validation

validation

3. Rule learning and prediction

Phase 4: Rule mining (phase4_rule_mining.py)
reads named graph urn:codexm:graph:train only
1- and 2-atom rules: symmetry, inverse, subrelation, chain, reversed chain
support · standard conf · PCA conf · head coverage · lift · Wilson bound
hub caps, redundancy filter, 90/10 stability check

Rule threshold review on validation
(phase4_rule_evaluation.py, phase4_compare_runs.py)
min PCA 0.10, min support 10; 6 rules excluded
→ frozen 160-rule set (33 head relations)

160 rules

Phase 5: Candidate generation (phase5_predict.py, one round)
apply rules to training facts: 212,342 rule×pair records
− 65,813 already in training · − 27,855 duplicates merged
= 184,487 unique candidate triples (with groundings)

Ranking V1
score = max Wilson lower bound
ties: PCA conf → #rules → subject ID → object ID

Ranking V2 (phase5_rank_v2.py)
same score
ties: PCA conf → distinct body matches → #rules → IDs

Per-relation cap
100 (baseline, outputs/phase5) · 250 · 500 · 1,000
(cap_experiment.py generate; baseline never rewritten)

cap 100

8 configurations

cap-100 V1/V2

4. Evaluation and output

Validation-based configuration selection
rule A1 frozen before validation: maximise recall subject to
Wilson lower bound of precision ≥ cap-100 bound − 0.05
8 configurations → selected cap 1,000 · V2

Prediction explorer (phase5_ui_build.py)
graph view, prediction table, entity search, filters,
provenance panel, V1 vs V2 comparison

Phase 6: test evaluation of cap-100 V1 and V2
(phase6_evaluate.py — first use of the test split)
TP 257 / 278 · FP 13 / 13

Final test evaluation of cap 1,000 · V2
(cap_experiment.py test — second use of the test split)
23,772 predictions · TP 927 · FP 73 · unlabelled 22,772

Metrics: benchmark precision TP/(TP+FP) · recall TP/|positives|
F1 = 2TP/(2TP+FP+FN) · positive hit rate TP/|predictions|
unlabelled predictions are not counted as false

Cap-comparison dashboard (cap_dashboard_build.py)
validation charts for all caps, validation table,
test table (cap-100 V1/V2, cap-1,000 V2)

Saved outputs and reproducibility records
predictions.json, CSV/JSON results, Markdown reports,
SHA-256 manifests, overwrite refusal, unit tests

Proposed future extensions (not implemented)

ontology-aware scoring · longer rules and constants · calibrated confidence
filtered MRR / Hits@K ranking protocol · embedding and neural baselines

Legend

- Interactive interface
- Training data / training-only processing
- Implemented processing stage
- Validation: rule review and cap selection
- Proposed, not implemented
- Test: evaluation only (used twice)